

Découvrir les notions de base de la programmation

Exercice 1 : Conversions entre systèmes de numération

- 1). Convertissez le nombre décimal 45 en binaire.
- 2). Convertissez le nombre binaire 101101 en décimal.
- 3). Convertissez le nombre décimal 78 en octal.
- 4). Convertissez le nombre octal 56 en décimal.
- 5). Convertissez le nombre décimal 255 en hexadécimal.
- 6). Convertissez le nombre hexadécimal 1A en décimal.
- 7). Convertissez le nombre binaire 111001 en octal.
- 8). Convertissez le nombre octal 34 en binaire.

Exercice 2 : Code ASCII

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

- 1). Quel est le code binaire du "a" minuscule en ASCII ?
- 2). Traduire en utilisant le convertisseur binaire – décimal, le message suivant : 01000010 01010010 01000001 01010110 01001111 00100001.

Exercice 3 : Algorithmes

1. Écrire un algorithme qui demande un nombre entier et affiche ce nombre d'une manière inversé.

2. Écrire un algorithme qui demande un nombre entier positif et affiche le nombre de chiffres qu'il contient.
3. Calculer la somme : $2 + 22 + 222 + 2222 + \dots$ (N termes)
Exemple : N=4
 $2 + 22 + 222 + 2222 = 2468$

Exercice 4 : Python

➤ **Détecteur d'anagramme dans une chaîne :**

Énoncé : Écrire une fonction Python `contient_anagramme(texte, motif)` qui prend deux chaînes de caractères en paramètres.

La fonction doit vérifier s'il existe, à l'intérieur du texte, une sous-chaîne contiguë qui est un anagramme exact du motif.

- Si une telle sous-chaîne existe, la fonction retourne `True`.
- Sinon, elle retourne `False`.

Note : Deux mots sont des anagrammes s'ils contiennent exactement les mêmes lettres avec les mêmes fréquences d'apparition, quel que soit leur ordre.

➤ **Le Premier Élément Non Répété**

Énoncé : Écrire une fonction `premier_unique(chaine)` qui prend en paramètre une chaîne de caractères et retourne le premier caractère qui ne se répète pas dans toute la chaîne.

Si tous les caractères se répètent ou si la chaîne est vide, la fonction doit retourner `None` (ou une chaîne vide "").

➤ **La Plus Longue Séquence Consécutive**

Énoncé : Écrire une fonction `plus_longue_sequence(liste_entiers)` qui prend en paramètre une liste de nombres entiers non triés et retourne la longueur de la plus longue séquence de nombres consécutifs présents dans la liste (l'ordre d'apparition initial dans la liste n'a pas d'importance).

Exemples d'exécution : `print(plus_longue_sequence([100, 4, 200, 1, 3, 2]))` # Doit afficher : 4

➤ Validation des Parenthèses et Symboles Imbriqués

Étape 1 : Le cas des structures centrées (Séquences Symétriques)

Dans cette première partie, on s'intéresse uniquement aux expressions dont la fermeture des symboles se fait de l'extérieur vers l'intérieur (structure en "miroir").

Énoncé : Écrire une fonction `symboles_valides_symetriques(chaine)` qui vérifie la cohérence des symboles d'ouverture (, [, { et de fermeture) ,] , } uniquement pour des expressions symétriques.

- **Exemples valides :** "`()`", "`{ }`", "`{ }`"
- **Exemples invalides :** "`()`" ou "`[ ]`"

Étape 2 : Généralisation à tous les cas de figure

L'approche symétrique de l'étape 1 ne permet pas d'analyser des expressions contenant des séquences adjacentes ou des imbrications complexes.

Énoncé : Faire évoluer le programme en écrivant la fonction `symboles_valides(chaine)` capable de valider n'importe quelle séquence de symboles, qu'ils soient imbriqués ou côte à côte.

- **Exemples qui doivent désormais être validés (True) :**
 - Séquences adjacentes : "`(){}`"
 - Mixte : "`{ }[ ]`"
- **Exemples invalides (False) :**
 - Mauvais ordre : "`[ ]`"
 - Symbole non fermé : "`()`"
 - Fermeture sans ouverture : "`)`"

➤ Le Calculateur de Moyennes Générales

Énoncé : On dispose d'une structure de données imbriquée représentant des étudiants et leurs notes par module avec des coefficients :

Python

```
etudiants = [  
    {  
        "nom": "Aymane",  
        "notes": {"Python": 15, "Algorithmique": 12, "Réseau": 14},  
        "coefficients": {"Python": 3, "Algorithmique": 2, "Réseau": 1}  
    },  
    # ... d'autres étudiants  
]
```

Écrire une fonction qui :

1. Calcule la moyenne pondérée de chaque étudiant.
2. Identifie le "Major de promotion" (l'étudiant avec la meilleure moyenne).
3. Retourne une nouvelle liste triée par ordre décroissant de moyenne.

➤ Le Système de Gestion d'un Inscription / Terminal de Billetterie

Énoncé : Vous devez concevoir la logique d'un système de réservation de billets pour un évènement (ex: une conférence ou un atelier technique). Le système s'articule autour de deux classes : Billet et GestionnaireBilletterie.

1. La classe *Billet*

Chaque billet possède :

- Un **identifiant unique** (ex: "B-101", "B-102"...).
- Le **nom du participant** (chaîne de caractères).
- La **catégorie** (ex: "Standard" ou "VIP").
- Un **prix de base** (ex: 100 DH pour Standard, 250 DH pour VIP).
- Un statut **est_valide** (Booléen, initialisé à True).
- **Méthode à implémenter :** utiliser() : Marque le billet comme utilisé (est_valide = False). Si le billet a déjà été utilisé, la méthode doit afficher un message d'erreur ou lever une alerte (ex: "*Attention : Billet déjà composté/utilisé !*").

2. La classe *GestionnaireBilletterie*

Cette classe gère l'ensemble des billets émis et la capacité maximale de la salle.

Attributs :

- capacite_max (nombre entier).
- billets (liste ou dictionnaire de tous les billets générés).

Méthodes :

1. **emettre_billet(nom_participant, categorie) :**
 - Vérifie s'il reste de la place dans la salle ($\text{len}(\text{billets}) < \text{capacite_max}$).
 - Si oui, génère un nouvel objet Billet avec un identifiant automatique, l'ajoute à la liste et le retourne.
 - Si la salle est pleine, affiche un message d'erreur et ne crée pas de billet.
2. **valider_entree(id_billet) :**
 - Recherche le billet correspondant à l'identifiant.
 - Si trouvé, appelle sa méthode utiliser().
 - Si non trouvé, affiche un message d'erreur ("*Billet inconnu*").
3. **calculer_recette_totale() :**

- Calcule et retourne le montant total généré par l'ensemble des billets émis.

➤ **Gestion des prêts (bibliothèque)**

1. Créer une classe abstraite Pret contenant les attributs nom_utilisateur, nb_livres_empruntes et jours_retard.
2. Ajouter dans cette classe une méthode concrète est_en_retard() qui retourne True si le nombre de jours de retard est supérieur à 0, sinon False.
3. Déclarer les méthodes abstraites suivantes dans la classe Pret : peut_emprunter(), calculer_penalite() et type_utilisateur().
4. Créer une classe Etudiant qui hérite de Pret.
 - a. Implémenter la méthode peut_emprunter() pour retourner False si le nombre de livres empruntés est supérieur ou égal à 3 ou si le retard dépasse 5 jours, sinon retourner True.
 - b. Implémenter calculer_penalite() pour calculer une pénalité égale à 2 DH multiplié par le nombre de jours de retard, et ajouter 10 DH supplémentaires si le retard dépasse 10 jours.
 - c. Implémenter type_utilisateur() pour retourner "Etudiant".
5. Créer une classe Professeur qui hérite de Pret.
 - a. Implémenter peut_emprunter() pour refuser uniquement si le nombre de livres empruntés est supérieur ou égal à 10.
 - b. Implémenter calculer_penalite() pour retourner 1 DH multiplié par le nombre de jours de retard, et 0 si aucun retard.
 - c. Implémenter type_utilisateur() pour retourner "Professeur".
6. Créer une classe Externe qui hérite de Pret.
 - a. Implémenter peut_emprunter() pour retourner False si le nombre de livres est supérieur ou égal à 2 ou si le nombre de jours de retard est supérieur à 0.
 - b. Implémenter calculer_penalite() pour calculer 5 DH multiplié par le nombre de jours de retard, et doubler cette pénalité si le retard dépasse 3 jours.
 - c. Implémenter type_utilisateur() pour retourner "Externe".
7. Dans le programme principal, créer trois objets de types différents avec des valeurs de test.
8. Pour chaque objet, afficher le type d'utilisateur, vérifier s'il peut emprunter, indiquer s'il est en retard et afficher la pénalité.
9. Pourquoi est_en_retard() est dans la classe abstraite ?